C Programming

ASSIGNMENT 1

CO1: *Construct structured C programs using constants, variables, data types and preprocessor directives to address basic computational tasks – structure of a C program, preprocessor directives, constants, variables, data types, type conversion, formatted input/output and the compilation process.*

Assignment Title: Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative (Total Marks - 50)

Submission Date: 26/10/2026

Bloom's Levels Assessed: K3 – Apply, K4 – Analyse, K5 – Evaluate (the level of each task is stated under the task heading)

Scenario — Grama Jyothi Rural Electricity Co-operative

Grama Jyothi is a village electricity co-operative that bills about 4,000 households every month. A meter reader records the previous and current readings of each house. The office computes the units consumed and the bill, which has a fixed charge and an energy charge that depends on slabs of consumption (for example: the first 100 units free, the next 100 units at Rs 2.25 per unit, and every unit above 200 at Rs 4.50).

The present spreadsheet has produced wrong bills: some bills are off by a few paise, a few households received a negative bill when their meter rolled over from 65535 to 0, and the tariff was typed in many places so that a tariff revision was applied only partly.

The number of readings processed may grow:

1 village (4,000 houses) → 1 taluk (60,000 houses) → 1 district (800,000 houses)

You are required to analyse the problem, choose suitable constants, data types and preprocessor features, implement the billing program in C and validate it, so that every bill is correct to the paisa.

Design Challenge:

The co-operative is moving to smart meters that report every 15 minutes and to a time-of-day tariff.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Data 4 Marks

Bloom's Level: K4 – Analyse

Analyse the billing problem and identify every item of data the program must store and compute.

Discuss:

- The inputs (previous reading, current reading, household number, category) and the outputs (units, energy charge, fixed charge, total)
- Which values never change during a run and should be constants
- Which values can be negative, fractional or very large
- The rules every correct bill must satisfy (for example: a bill is never negative; the total equals the sum of its parts)
- Any assumptions you make

You are expected to **justify each data item**, not merely list names.

Alternative Concept and Complexity Analysis

If the meter readings were read and stored as text (character strings such as "01530") instead of integers, what would the solution look like? Analyse the time needed to convert and compare each reading (proportional to the number of digits d) and the space used per reading in both representations.

Evidence Expected

A clear analysis of the inputs, outputs, constants, rules and assumptions of the billing problem.

TASK 2 – Choose the Data Types 4 Marks

Bloom's Level: K4 – Analyse

Select a C data type for every data item identified in Task 1.

Discuss:

- The type chosen (char, short, int, long, long long, unsigned, float, double) and its size using sizeof on your system
- The range of each type using limits.h and float.h, and whether it is sufficient for a district
- Why money should not be stored as float rupees, and the alternative of storing paise in an integer
- What happens to a 16-bit unsigned meter register after 65535

Show one example where **float gives a wrong total** when 4,000 bills are added, and how your choice avoids it.

Complete the Code

Complete TODO 1 below without using if–else, and execute the program.

```
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
   // TODO 1: Return the units consumed, correct even when the register has rolled over
   //   (current < previous); use unsigned 16-bit arithmetic instead of an if-else

}

int main(void)
{
   printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
         sizeof(meter_t), (unsigned)USHRT_MAX);
   printf("Consumption: %u units\n", consumption(1200, 1530));
   printf("Consumption: %u units\n", consumption(65400, 120));   /* rolled over */
   return 0;
}
```

Expected output:

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

Explain why the second result is 256 and not a negative number.

Alternative Concept and Complexity Analysis

Suppose money were stored as double rupees instead of long paise. What would the billing solution be, and where would rounding be applied? Analyse the time per addition (O(1) in both) and the space per value, and compare how the rounding error grows when n bills are added.

Evidence Expected

A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

TASK 3 – Define the Constants 3 Marks

Bloom's Level: K3 – Apply

Define the tariff values and limits as constants so that a tariff revision is made in one place only.

Discuss:

- The use of #define, const variables and enum for different kinds of constants
- The difference between a #define constant and a const variable (type checking, scope, debugging)
- Naming conventions for constants

Complete the Code

Complete TODO 1 using only the macros defined at the top, and execute the program.

```
#include <stdio.h>

#define SLAB1_UNITS 100          /* first 100 units          */
#define SLAB2_UNITS 200          /* units 101 to 200         */
#define RATE1_PAISE 0            /* first slab is free       */
#define RATE2_PAISE 225          /* Rs 2.25 per unit         */
#define RATE3_PAISE 450          /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L   /* Rs 50 fixed charge       */

/* Returns the bill in paise, so that no rounding error can occur. */
long compute_bill(int units)
{
   long paise = FIXED_CHARGE_PAISE;
   // TODO 1: Add the energy charge slab by slab using the macros: units above
   //   SLAB2_UNITS at RATE3_PAISE, units from SLAB1_UNITS+1 to SLAB2_UNITS at
   //   RATE2_PAISE, and the rest at RATE1_PAISE

   return paise;
}

int main(void)
{
   int tests[] = {0, 100, 150, 200, 350};
   int n = sizeof tests / sizeof tests[0];
   for (int i = 0; i < n; i++) {
      long p = compute_bill(tests[i]);
      printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
   }
   return 0;
}
```

Expected output:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

Change RATE2_PAISE to 230 and check that every bill changes consistently.

Alternative Concept and Complexity Analysis

If the tariff constants were written as const variables or as an enum instead of #define, what would change in the program? Compare type checking, visibility in the debugger and the memory used (a #define occupies no storage; a const variable may), and state whether the running time is affected.

Evidence Expected

A single, well-named set of constants with a justified choice between #define, const and enum.

TASK 4 – Use Macros Safely 4 Marks

Bloom's Level: K4 – Analyse

Write macros for small repeated calculations (for example: converting rupees to paise, finding the larger of two values).

Discuss:

- How a macro is expanded by the preprocessor, shown using gcc -E
- Why SQUARE(x) written as x*x gives a wrong result for SQUARE(a+1), and how parentheses fix it
- Why MAX(a, b) can evaluate an argument twice (for example MAX(i++, j))
- When a function is a better choice than a macro

Alternative Concept and Complexity Analysis

If the MAX and SQUARE macros were replaced by static inline functions, what would the solution be? Analyse the time cost of a call versus an expanded macro, and the code size (space) when the macro is used k times in the program, since a macro is copied at every use.

Evidence Expected

Correct, fully parenthesised macros with gcc -E evidence and an explanation of each macro pitfall.

TASK 5 – Organise the Program with Header Files and Conditional Compilation 3 Marks

Bloom's Level: K3 – Apply

Organise the constants and function prototypes into a header file and add a debugging mode.

Discuss:

- The structure of a C program: header files, global declarations, main() and other functions
- Include guards (#ifndef … #define … #endif) and why they are needed
- Conditional compilation with #ifdef DEBUG to print intermediate values only when compiled with gcc -DDEBUG
- The difference between #include <file> and #include "file"

Alternative Concept and Complexity Analysis

If debugging output were controlled by a run-time variable (if (debug) printf(…)) instead of #ifdef DEBUG, what would the solution be? Analyse the time cost of the extra test in every call and the space taken by debug code that remains in the executable in each approach.

Evidence Expected

A header file with include guards, a working DEBUG mode and an explanation of the program structure.

TASK 6 – Read and Print Data Correctly 4 Marks

Bloom's Level: K3 – Apply

Design the input and output of the program using scanf and printf.

Discuss:

- The correct format specifier for each data type (%d, %u, %ld, %lld, %f, %lf, %c, %s)
- What happens when a wrong specifier is used (for example %d for a long), shown by a test
- How to check the return value of scanf to detect invalid input such as letters
- How to print rupees and paise from an integer amount in paise (for example Rs 162.50)

Alternative Concept and Complexity Analysis

If each input line were read with fgets() and then converted with sscanf() or strtol() instead of reading directly with scanf(), what would the solution be? Analyse the time to process a line of length L and the buffer space needed, and compare how each approach recovers from invalid input.

Evidence Expected

Correct format specifiers, input validation using the return value of scanf, and a correctly formatted bill.

TASK 7 – Control Type Conversion and Precision 4 Marks

Bloom's Level: K4 – Analyse

Identify every place in the program where values of different types are combined.

Discuss:

- Implicit conversion (integer promotion and the usual arithmetic conversions)
- Integer division (for example units / 100) and where it gives a wrong result
- Explicit casting, and the effect of casting a large long to int
- Rounding rules when converting a fractional charge to paise

Show the output of at least **three expressions** that give an unexpected result, and explain each one.

Alternative Concept and Complexity Analysis

If all calculations were done in float and rounded only once at the end, instead of in integer paise, what would the solution be? Measure and analyse how the error in the total grows with the number of bills n, and compare the time and space per value of each approach.

Evidence Expected

Identified conversion points, three demonstrated surprises with explanations, and a consistent rounding rule.

TASK 8 – Trace the Compilation Process 4 Marks

Bloom's Level: K3 – Apply

Follow your program through the stages of compilation.

Discuss:

- The output of each stage: gcc -E (preprocessed), gcc -S (assembly), gcc -c (object file) and the linked executable
- What the preprocessor removed or replaced in your program
- The difference between a compile-time error, a link-time error and a run-time error, with one example of each from your work
- What the warnings from gcc -Wall -Wextra told you and how you resolved them

Alternative Concept and Complexity Analysis

If the program were split into several .c files (input.c, tariff.c, report.c) with a shared header and compiled separately, instead of one file, what would the build process be? Analyse the compile time after a change to one file in both approaches, and the space used by the object files.

Evidence Expected

Evidence of each compilation stage and correctly classified examples of errors and warnings.

TASK 9 – Implement and Test the Billing Program 4 Marks

Bloom's Level: K3 – Apply

Implement the complete billing program and test it.

Test:

- Normal bills in each slab
- Boundary cases: exactly 100 units, exactly 200 units, zero units
- A meter that has rolled over from 65535 to 0
- Invalid input: letters, a negative reading, a current reading less than the previous one without roll-over
- A small set of 5 bills whose total you calculate by hand

Alternative Concept and Complexity Analysis

If the slab limits and rates were stored in two arrays and the bill computed by a loop over the slabs, instead of fixed if statements, what would the solution be? Analyse the time per bill for s slabs (O(s) versus a fixed number of tests) and the extra space, and explain which design is easier to change.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

TASK 10 – Compare Expectations with Execution 4 Marks

Bloom's Level: K4 – Analyse

Compare what you expected from your type and constant design with what actually happened.

Discuss:

- Whether the float and the paise versions of the total for 4,000, 60,000 and 800,000 bills agree, and by how much they differ
- Whether the int, long and long long totals overflow at district scale
- How sizeof and the ranges on your system compare with the minimum sizes guaranteed by the C standard
- Reasons for any difference

Alternative Concept and Complexity Analysis

If the district total were accumulated in a double instead of a long long, what would happen at 800,000 bills? Analyse when each type loses accuracy (overflow versus precision loss), and compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the final set of data types, constants and preprocessor features for the co-operative.

Discuss:

- The types you recommend and why
- How a tariff revision will be carried out
- The limitations of your design
- Whether the design remains correct at district scale

Your recommendation must be based on **evidence from your own experiments**.

Alternative Concept and Complexity Analysis

If the tariff were read from a configuration file when the program starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read t tariff values and the space they occupy, against the cost of recompiling and redistributing the program after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The co-operative is installing **smart meters** that send a reading in watt-hours every **15 minutes** (35,040 readings per house per year). A **time-of-day tariff** applies: units used between 6 PM and 10 PM cost 20% more. The board also wants the tariff values to be changeable **without recompiling** the program.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which data types are no longer sufficient (watt-hours, yearly totals)
- Which constants can no longer be fixed with #define

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether the original types, constants and macros remain appropriate. Use calculations of the largest values that can now occur.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe and implement the revised design (for example: wider integer types, reading tariff values at run time instead of #define, a peak-hour rule).

Evidence Expected

An appropriate design response with working code.

D. Evidence-Based Defence – 2 Marks

Defend your decision using the largest values computed, test results and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating calculations, tests and the changed requirements.

MARK DISTRIBUTION

Task	**Bloom's Level**	**Marks**
Task 1 – Problem Analysis	K4 – Analyse	4
Task 2 – Data Types	K4 – Analyse	4
Task 3 – Constants	K3 – Apply	3
Task 4 – Macros	K4 – Analyse	4
Task 5 – Header Files & Conditional Compilation	K3 – Apply	3
Task 6 – Formatted Input/Output	K3 – Apply	4
Task 7 – Type Conversion & Precision	K4 – Analyse	4
Task 8 – Compilation Process	K3 – Apply	4
Task 9 – Implementation & Testing	K3 – Apply	4
Task 10 – Expectation vs Execution	K4 – Analyse	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

SUBMISSION REQUIREMENTS

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.
- **Assignment report** containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

ASSESSMENT VERIFICATION

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.